

CONFIDENTIAL — INTERNAL PRESENTATION

NexoraDB

API | API Contracts & Data Structures قراردادهای

Standard / استاندارد

C++20

namespace

nexora::core / graph / query

Version / نسخه

1.0.0

توضیح	DESCRIPTION
تمام توابع عمومی DocEngine این ساختار را برمی‌گردانند. هر تیمی که با DocEngine کار می‌کند باید این ساختار را بشناسد.	All public DocEngine functions return this struct. Every team working with DocEngine must understand this type.

NEXORA::CORE

struct DBResult

```

struct DBResult { bool success = false; // آیا عملیات موفق بود؟
std::string data = ""; // مقدار بازگشتی (doc_id / BSON / count)
std::string error_msg = "";
// Factory methods static DBResult Ok(std::string payload = "");
static DBResult Err(std::string msg); };

```

عملیات	data در حالت موفق	مثال
InsertOne	UUID سند جدید	"...-a1b2c3d4"
InsertMany	JSON آرایه IDها	["id1","id2","id3"]
FindById	محتوای BSON سند	{"id":"...","name":"Ali_"}
FindMany	JSON آرایه اسناد	[{doc2},{doc1}]
UpdateById	تعداد سند تغییر یافته	"1"
DeleteById	حذف شد یا نه	"0" یا "1"
Count	تعداد به صورت string	"42"
CreateCollection	پیام تأیید	"Collection 'users' created"

● مدیریت Collection / Collection Management

```
DBResult CreateCollection(string name, optional<SchemaDefinition> schema  
= nullopt)
```

```
"data = "Collection 'X' created"
```

Collection Mgmt

یک collection جدید می‌سازد. اگر Schema داده شود، اعتبارسنجی فعال می‌شود. اگر collection از قبل وجود داشته باشد، خطا برمی‌گرداند.

Creates a new collection. If schema is provided, validation is enabled. Returns error if collection already exists.

```
DBResult DropCollection(string collection_name)
```

```
"data = "Collection 'X' dropped"
```

Destructive

⚠ برگشت‌ناپذیر. تمام اسناد، ایندکس‌ها و FK‌های آن collection حذف می‌شوند. / Irreversible.
.Removes all documents, indexes, and FKs

```
bool CollectionExists(string name) const
```

بررسی سریع O(1) وجود Quick O(1) existence check. / collection.

```
vector<string> ListCollections() const
```

لیست نام تمام collections. مثال: `["users", "posts", "messages"]`

● عملیات CRUD

```
DBResult InsertOne(string collection, string bson_document)
```

```
data = doc_id (UUID v4)
```

یک سند BSON درج می‌کند. اگر `id_` وجود نداشته باشد، UUID v4 اختصاص می‌یابد. Schema Validation و FK Check قبل از نوشتن انجام می‌شود.
Inserts one BSON document. Auto-generates UUID v4 if `_id` is absent. Schema + FK validated before write.

```
DBResult InsertMany(string collection, vector<string> docs)
```

```
'data = '["id1","id2","id3"]
```

درج اتمیک چند سند با RocksDB WriteBatch. یا همه موفق می‌شوند یا هیچکدام. / Atomic multi-insert via WriteBatch. All-or-nothing

```
DBResult FindById(string collection, string doc_id) const
```

```
data = BSON string
```

O(1)

دسترسی مستقیم با کلید — سریع‌ترین عملیات خواندن. / Direct key access — fastest read operation.

```
DBResult FindMany(string collection, Condition cond, uint32_t limit=0, uint32_t skip=0) const
```

```
'data = ' [{doc1},{doc2}]
```

جستجو با شرط. اگر فیلد مورد نظر ایندکس داشته باشد، Index Scan انجام می‌شود؛ در غیر این صورت Full Scan. / Condition-based search. Uses index scan if available, otherwise full scan

```
DBResult UpdateById(string collection, string doc_id, UpdateSpec spec)
```

Read-Modify-Write اتمیک. ایندکس‌های مرتبط به‌روز می‌شوند. FK در صورت تغییر فیلد مرتبط بررسی می‌شود.

```
DBResult UpdateMany(string collection, Condition cond, UpdateSpec spec)
```

به‌روزرسانی همه اسناد منطبق با شرط. data = تعداد سند تغییر یافته.

```
DBResult DeleteById(string collection, string doc_id)
```

حذف سند و پاکسازی ایندکس‌ها. `"data="1` اگر حذف شد، `"0"` اگر وجود نداشت.

Foreign Key و Index ●

```
DBResult CreateIndex(string collection, IndexDefinition def)
```

ایندکس جدید می‌سازد و تمام اسناد موجود را بازنویسی می‌کند (rebuild). پس از ایجاد، `FindMany` به صورت خودکار از آن استفاده می‌کند.

```
DBResult AddForeignKey(string collection, ForeignKeyDefinition fk)
```

کلید خارجی تعریف می‌کند. از این پس در `InsertOne/UpdateById` مقدار فیلد در `collection` مرجع تأیید می‌شود.

```
vector<ForeignKeyDefinition> GetForeignKeys(string collection) const
```

★ **GraphEngine** از این تابع استفاده می‌کند تا `edge types` را کشف کند. هر `FK` = یک نوع `edge` در گراف. / `GraphEngine uses this to discover edge types automatically`.

Internal API برای GraphEngine / Internal API ●

```
void IterateCollection(string collection, DocumentCallback fn, uint32_t batch=100) const
```

Graph-Only API

فقط برای **GraphEngine** در **Startup**. برای هر سند فراخوانی می‌شود. اگر `callback` مقدار `false` برگرداند، iteration متوقف می‌شود. `fill_cache=false` برای `bulk scan`.

```
engine.IterateCollection("users", [&](const std::string& id, const
std::string& bson) -> bool { graph.addNode(id, "User"); return
true; // ادامه بده });
```

```
int64_t GetCollectionSize(string collection) const
```

O(1) از شمارنده داخلی. GraphEngine برای pre-allocate حافظه استفاده می‌کند:

```
user_nodes_.reserve(engine.GetCollectionSize("users"))
```

Transactions (ACID) ●

```
// الگوی استفاده / Usage Pattern auto tx = engine.BeginTransaction(); //
شروع if (!tx) { /* handle error */ } engine.InsertOneTx(*tx, "messages",
msg_bson); engine.UpdateByIdTx(*tx, "users", user_id, spec); auto result
= engine.CommitTransaction(*tx); // در صورت خطا:
engine.RollbackTransaction(*tx);
```

توضیح	تابع
شروع transaction جدید — Pessimistic Locking	<code>BeginTransaction()</code>
تأیید — تغییرات ذخیره می‌شوند	<code>CommitTransaction(tx)</code>
عقب‌گرد — تغییرات دور انداخته می‌شوند	<code>RollbackTransaction(tx)</code>
InsertOne داخل transaction	<code>InsertOneTx(tx, ...)</code>
UpdateById با lock GetForUpdate	<code>UpdateByIdTx(tx, ...)</code>
DeleteById داخل transaction	<code>DeleteByIdTx(tx, ...)</code>
خواندن با دید به uncommitted changes	<code>FindByIdTx(tx, ...)</code>

NEXORA::QUERY

struct Condition

```
struct Condition { // — Leaf fields — std::string field; // "age" /  
"address.city" Op op; // EQ, GT, IN, REGEX... std::string value; // "18", "ali"  
(string-encoded) ValueType value_type; // String, Int64, Float64, Bool, Null  
vector<string> values; // برای IN / NIN // — Composite fields — LogicOp  
logic; // AND, OR, NOR, NOT vector<Condition> sub_conditions; // — Helpers —  
bool IsLeaf() const; // شرط ساده bool IsEmpty() const; // match اسناد همه bool  
IsComposite() const; // شرط ترکیبی // — Factory — static Condition  
Leaf(field, op, value, value_type); static Condition And(vector<Condition>  
subs); static Condition Or(vector<Condition> subs); static Condition In(field,  
values, negate=false); static Condition Nor(vector<Condition> subs); };
```

enum class Op

GT =2

greater / بزرگتر

NEQ =1

not equal / نابرابر

EQ =0

equal / برابر

LTE =5less-equal / $\geq$ **LT** =4

less / کوچکتر

GTE =3greater-equal / $\leq$ **EXISTS** =8

field exists / وجود فیلد

NIN =7

not in / غیرعضو

IN =6

in list / عضو لیست

CONTAINS =11

substring / حاوی

STARTS =10

prefix / پیشوند

REGEX =9

regex pattern / الگو

enum class LogicOp & ValueType

NOR

هیچکدام نباید صادق

OR

حداقل یکی کافی است

AND

همه باید صادق باشند

NOT

negate sub_conditions[0]

Float64

اعشاری

Int64

عدد صحیح

String

رشته

Null

تهی

Bool

بولین

UpdateSpec — Update Operations

UpdateSpec — عملیات به روزرسانی

04

NEXORA::QUERY

struct UpdateSpec

```
// Builder Pattern: UpdateSpec spec; spec.Set("bio", "Hello World")
.Inc("likes", "1", UpdateValueType::Int64) .Push("tags", "sport")
.TouchDate("updated_at"); // = CurrentDate
```

عملیات	معادل MongoDB	رفتار	مثال value
Set	set\$	تنظیم/ایجاد فیلد	"Hello World"
Unset	unset\$	حذف فیلد از سند	—
Inc	inc\$	افزودن delta (منفی = کاهش)	"1" یا "5-"
Mul	mul\$	ضرب مقدار عددی	"1.1"
Min	min\$	اگر مقدار جدید کمتر است جایگزین کن	"100"
Max	max\$	اگر مقدار جدید بیشتر است جایگزین کن	"0"
Rename	rename\$	تغییر نام فیلد	"new_name"
CurrentDate	currentDate\$	Unix timestamp (ms)	—
Push	push\$	افزودن به آرایه	"sport"
PushAll	each\$	افزودن چند عنصر	در []values
Pull	pull\$	حذف از آرایه	"sport"
AddToSet	addToSet\$	افزودن اگر وجود نداشت	"tag"
Pop	pop\$	"1"=آخر، "1-="اول	"1"

NEX FILE.

NodeRecord (32 bytes)

```
struct NodeRecord { // 32 bytes fixed uint64_t dense_id; // Dense-ID (index)
uint32_t type_id; // از نوع TypeRegistry uint32_t flags; //
ACTIVE/DELETED/IMPLICIT uint64_t out_degree; // پال‌های خروجی uint64_t
in_degree; // پال‌های ورودی }; // offset = dense_id * 32 → O(1)
```

NEXR FILE.

EdgeRecord (32 bytes)

```
struct EdgeRecord { // 32 bytes fixed uint64_t edge_id; // EdgeId uint64_t src;
// DenseId uint64_t dst; // DenseId uint32_t type_id; // نوع رابطه
uint32_t flags; // ACTIVE/DELETED }; // offset = edge_id * 32 → O(1)
```

GRAPH.WAL

WalRecord (56 bytes)

```
struct WalRecord { uint64_t seq; // ترتیبی بکتا int64_t timestamp_ms; // unix
ms WalOpType op; // نوع عملیات uint64_t node_or_edge_id; uint64_t src_id; //
uint64_t dst_id; // برای edges uint32_t type_id; uint32_t flags; bool applied; //
اعمال شده؟ RAM روی };
```

WalOpType & Direction & GraphState

AddEdge=3

edge اضافه شد

RemoveNode=2

node حذف شد

AddNode=1

node اضافه شد

Rollback=12

عقبگرد

Commit=11

تأیید تراکنش

RemoveEdge=4

edge حذف شد

Both

هر دو جهت

In

پال‌های ورودی

Out

پال‌های خروجی

Corrupted

Dirty

Clean

API — LiveGraph عمومی برای تیم الگوریتم

پیچیدگی	خروجی	ورودی	متد
$O(1)$	DenseId	ExtId, string	<code>addNode(ext_id, type)</code>
$O(\text{deg})$	bool	ExtId	<code>removeNode(ext_id)</code>
$O(1)$	bool	ExtId	<code>hasNode(ext_id)</code>
$O(1)$	DenseId	ExtId	<code>getDenseId(ext_id)</code>
$O(\log \text{deg})$	EdgeId	ExtId, ExtId, string	<code>addEdge(src, dst, type)</code>
$O(\log \text{deg})$	bool	EdgeId	<code>removeEdge(edge_id)</code>
$O(\text{limit})$	<vector<DenseId	DenseId, Direction, Typed	<code>neighbors(id, dir, type, limit)</code>
$O(\text{limit})$	<vector<ExtId	ExtId, Direction, string	<code>neighborsExt(ext, dir, type)</code>
$O(V)$	void	callback	<code>forEachNode(fn)</code>
$O(E)$	void	callback	<code>forEachEdge(fn)</code>
$O(1)$	GraphStats	—	<code>() stats</code>
$O(V+E)$	bool	—	<code>() compact</code>

CONTRACT

The FastAPI team communicates with C++ via pybind11. No logic should reside in Python — only type conversion and routing.

قرارداد

تیم FastAPI از طریق pybind11 با ++C ارتباط برقرار می‌کند. هیچ منطقی در Python نباید باشد — فقط routing و تبدیل type.

```
# Python side (after pybind11 binding) import nexoradb engine =
nexoradb.DocEngine("/var/data/nexoradb") graph = nexoradb.LiveGraph(engine) #
Document operations result = engine.insert_one("users", '{"username":"ali"}')
if result.success: doc_id = result.data # "uuid-v4-string" result =
engine.find_many("users", {"age": {"$gt": 18}}, limit=10) docs = result.data #
'[{"...},{...}]' JSON string # Graph operations graph.add_node("u_001", "User")
graph.add_edge("u_001", "u_002", "FOLLOWS") followers =
graph.neighbors_ext("u_001", "in", "FOLLOWS", limit=100) # Algorithm execution
result = graph.run_lock("CommonFollowers", ["u_001", "u_002"]) future =
graph.submit_job("PageRank", ["20", "0.85"])
```